St Mary's University Twickenham London

CPS7002 – Software Design and Development – Assessment Brief

Module Code:	CPS7002
Module Title:	Software Design and Development
Module Convenor:	Elisabetta Canetta
Module Level:	7

Assessment Number:	1
Assessment Title:	Smart Maritime Port Operations Optimisation System (SMPOOS)
Assessment Weight:	60%
Assessment Individual/Group:	Individual
Assessment Type:	Software Artefact with Report
Assessment Time/Word Count Restrictions:	1 software artefact 3000 words report
Assessment Time/Word Count Limit Consequences:	It is essential that assignments keep within the time/word count limit stated above. Any work beyond the maximum time/word length permitted will be disregarded and not accounted for in the final grade.

Issue Date:	05th March 2026
Hand in Date:	03rd June 2026
Planned Feedback Date:	Within 3 working weeks
Mode of Submission:	Online via Moodle
Number of copies to be submitted:	1 copy of each of the following: • a report in pdf format • a zip file containing your software artefact

Author:	Elisabetta Canetta
Internal Moderator:	Prins Butt
Moderation Date:	19th February 2026

Table of Contents

Introduction 3
Requirements 3
Tasks 4
Environment and Tools 6
Submission 7
Assessment Criteria 8
Learning Outcomes 9
Regulations, Policies, and Guidelines 11

Introduction

HarbourView International Port, one of the busiest logistics hubs in Europe, is planning to implement a Smart Maritime Port Operations Optimisation System (SMPOOS) to enhance operational efficiency, reduce vessel turnaround time, and improve safety in port environments.

The envisioned system should integrate:

- **Vessel tracking** using AIS (Automatic Identification System) data
- **Predictive analytics** for berth allocation and congestion forecasting
- **Environmental sensors** for weather, tide, and air-quality monitoring
- **Location-based notifications** for hazards, delays, or workflow changes sent to dock workers and port administrators
- **An interactive management dashboard** accessible across devices, showing operational KPIs, predictive insights, and automated alerts

The port authority seeks an analytical report and a software prototype evaluating the feasibility, architecture, risks, and ethical considerations of introducing SMPOOS.

Requirements

You have been hired as a software consultant to design and develop the Smart Maritime Port Operations Optimisation System (SMPOOS).

The system must be:

- **Scalable**, to support large and growing vessel traffic
- **Maintainable**, allowing updates as port operations evolve
- **Secure**, due to the sensitivity of maritime logistics data

The system must integrate with existing port data sources, such as:

- Vessel arrival schedules
- Berth allocation data
- AIS (Automatic Identification System) feeds
- Environmental sensor data (tides, weather, air quality)
- Workforce activity logs
- Port asset and equipment databases

Use the 4 .csv files on Moodle (**smpoos_locations.csv, smpoos_routes.csv, smpoos_notifications.csv, smpoos_users.csv**) as data sources:

- **smpoos_locations.csv**
  Columns include:
  - location_id
  - name (Berth number + Area letter)
  - type (Berth, Storage Yard, Fuel Dock, etc.)
  - status (Active, Inactive, Under Maintenance)
  - capacity_tonnes

- **smpoos_routes.csv (shipping lanes / internal transit paths)**
  Columns include:
  - route_id
  - start_location / end_location
  - route_type (Shipping Lane, Tugboat Path, AGV Route, etc.)
  - distance_km
  - status

- **smpoos_notifications.csv (port alerts)**
  Columns include:
  - notification_id
  - alert_type (Weather Warning, Equipment Failure, etc.)
  - location_id
  - severity
  - message
  - timestamp (randomised past times)

- **smpoos_users.csv (port personnel)**
  Columns include:
  - user_id
  - name
  - role (Harbourmaster, Stevedore, etc.)
  - email
  - active

The system should fulfil the following requirements:

(a) **SMPOOS management**:

- **Administrative capabilities**
  Admin users must be able to:

  - Add, update, and remove port zones, berth areas, internal transport routes, and operational notifications via a secure interface
  - Modify vessel paths, loading/unloading routes, or restricted areas in real time (e.g., closed berths, hazardous areas, crane outages)
  - Use role-based access control (RBAC) to ensure only authorised personnel (harbourmaster, operations supervisors, safety officers) can modify operational data

- **Operational changes**
  The system should support:

  - Immediate reconfiguration of vessel and vehicle routes during temporary disruptions (e.g., storms, delays, equipment failures)
  - Automated recalculation of berth allocation based on new conditions

(b) **Real-time monitoring and alerts/notifications**:

- **Operational monitoring**
  The system must include monitoring of:

  - Vessel movements and estimated times of arrival/departure
  - Berth occupancy
  - Cargo-handling activity
  - Environmental updates (wind speed, tide levels, visibility)
  - System health and event logs

- **Notification engine**
  The system must deliver alerts to users based on:

  - **Location** (e.g., hazard near Berth 12)
  - **Role** (e.g., stevedores receive safety alerts, harbourmasters receive congestion warnings)
  - **Operational context** (delays, equipment faults, weather hazards, workflow changes)

- **Notification delivery**
  Notifications should be available via:

  - In-system notifications
  - Email

- **Admin controls**
  Admins must be able to:

  - Configure notification rules and thresholds (e.g., congestion warnings when tugboats exceed capacity, unsafe wind speed for crane operations)
  - Review a full audit trail of alerts, including timestamps, severity, and delivery status

(c) **Regulatory compliance**:

The system must comply with:

- **GDPR** and similar regulatory frameworks
- **Maritime safety regulations** (e.g., IMO data handling guidelines)
- **Port authority governance requirements**

It must also:

- Include user consent mechanisms for any personal or workforce data monitoring
- Support data access, portability, and deletion requests from users
- Maintain audit logs for all administrative actions, data modifications, and system access

(d) **Data analytics and reporting**:

The system must collect and analyse data such as:

- Most used berths or port zones
- Common vessel routes or internal transport paths
- Peak congestion times
- Equipment utilisation rates
- Environmental trends (e.g., tidal patterns, wind delays)

It must also:

- Provide visual analytics dashboards for port managers and planners
- Enable custom report generation, filtered by date, vessel type, cargo type, or port area
- Use analytics to suggest berth allocation optimisations, identify operational bottlenecks, and highlight safety or accessibility issues

(e) **Security and privacy**:

The system must include:

- Secure authentication
- Encryption of sensitive data, both in transit and at rest (e.g., encrypting AIS data streams)
- Access control policies aligned with port security protocols (e.g., protecting vessel manifests and logistical schedules
- Regular vulnerability scans, security audits, and patching

(f) **Management dashboard**:

A responsive administrative dashboard must provide:

- **Key dashboard widgets**
  - **Port status overview** (berth occupancy, vessel queue, environmental conditions)
  - **Vessel and vehicle activity**
  - **Notifications and alerts panel**
  - **Data analytics summaries** (performance KPIs, congestion heatmaps)

- **Customisation and interactivity**
  - Custom dashboard layouts based on user roles (e.g., harbourmaster vs. crane supervisor)
  - Interactive map overlays (e.g., berth layout, shipping lanes, restricted zones)
  - Filters, drill-downs, and historical playback of vessel movements

(g) **Scalability**:

To ensure long-term viability:

- Use a modular, layered architecture
- Support increased vessel data volume, more sensors, and more concurrent users
- Implement load balancing for AIS data processing and analytics services
- Use caching and stream-processing optimisations to maintain performance as port traffic grows

Tasks

You are required to develop a software system that meets the requirements stated above.

Software Artefact Criteria:

- Ensure the software solution is fully functional, capable of execution without errors
- Demonstrate highly effective object-oriented design and development techniques
- Prepare and include with the software artefact a clear and professional README.md file

Report Criteria:

- Provide a detailed summary of the implemented features, accompanied by justifications and supporting evidence, including relevant diagrams
- Detail the implementation process, supported by evidence and rationale
- Include evidence of version control to demonstrate the management of software changes
- Provide a thorough explanation and evaluation of the solution, supported by strong evidence
- Address professional ethics and practices.
- Include evidence of version control, showcasing exemplary development practices

Outline of the report

Title Page

- Project Title
- Course Title
- Date of Submission

Abstract (150-200 words)

- Brief overview of the project
- Key features implemented
- Summary of outcomes and evaluation

1. Introduction (300-400 words)

- Context and background of the project
- Objectives and scope
- Overview of technologies/tools used
- Structure of the report

2. Implemented Features (500-600 words)

- Detailed description of each feature
- Justification for inclusion (e.g., user needs, technical requirements)
- Supporting evidence (e.g., benchmarks, user feedback)
- Relevant diagrams (e.g., architecture, workflows)

3. Implementation Process (400-500 words)

- Step-by-step breakdown of development
- Tools and frameworks used
- Challenges encountered and how they were addressed
- Rationale behind key decisions

4. Version Control Evidence (300-400 words)

- Overview of version control system used (e.g., GitHub)
- Screenshots or logs of commits, branches, merges
- Explanation of how version control supported development
- Examples of good practices (e.g., commit messages, branching/merging)

5. Evaluation of the Solution (500-600 words)

- Functional and non-functional testing results
- Performance metrics
- Critical analysis of strengths and limitations
- Suggestions for future improvements

6. **Professional Ethics and Practices (200-300 words)**

- Ethical considerations (e.g., data privacy, accessibility)
- Compliance with relevant standards or regulations
- Responsible coding and documentation practices
- Reflection on professional conduct

8. Conclusion (200–300 words)

- Summary of achievements
- Reflection on learning outcomes
- Final thoughts on the project's impact

References

- A minimum of 12 literature sources properly formatted using Harvard referencing style
- Sources should be mainly primary sources (i.e., peer-reviewed articles published in good quality journals, books, technical reports with DOI number)

Appendices

- Additional diagrams
- Full version control logs
- Full codes

Environment and Tools

You are required to use the following tools:

- **PyCharm or VScode or Jupyter Notebook**: as your integrated development environment.
- **Python 3.11+**: as the standard python library.
- **Draw.io** (or equivalent tool): To create diagrams.
- **Microsoft Word** (or equivalent tool): To author your report and export as a PDF.
- **Adobe PDF Reader**: To view your final report which is saved as a PDF file.
- **Git Tools and GitHub**: for version control.

Additionally, the following libraries/modules may be imported and utilised:

1. csv
Used for reading/writing SMPOOS datasets such as:
- Port locations (berths/areas)
- Shipping lanes / internal transport routes
- Port notifications (alerts)
- Port personnel data

Useful for quick ingestion of the CSV files provided.

2. enum
Use for defining:
- User roles (Harbourmaster, Stevedore, etc.)
- Alert types (Weather Warning, Berth Closure, etc.)
- System states (Active, Restricted, Closed)

Enums improve readability and reduce errors in lookup logic.

3. datetime
Used for:
- Generating timestamps for alerts, events, and system logs
- Calculating vessel arrival/berth-occupancy durations
- Time-based filtering in analytics dashboards

4. matplotlib
A core library for creating static reports such as:
- Berth utilisation charts
- Congestion over time
- Environmental trend visualisations (wind, tide levels)

5. seaborn
Used for producing enhanced analytics visuals, such as:
- Heatmaps (e.g., busiest port zones)
- Trend lines for operational activity
- Distribution plots for vessel turnaround times

Seaborn builds on matplotlib to make charts more interpretable.

6. plotly
Used to produce rich, interactive visualisations, including:
- Interactive shipping-lane graphs
- Berth-occupancy timelines
- Vessel-movement scatter maps
- Congestion heatmaps

Plotly integrates well with Dash for interactive dashboards.

7. random
Useful for:
- Simulating vessel arrivals or delays
- Generating pseudo-user activity (test data)
- Creating stochastic scenarios for prototyping

8. math
Supports computational tasks such as:
- Distance calculations between port coordinates
- Basic optimisation logic for berth allocation
- Route-efficiency scoring

9. dash (Plotly Dash)
Essential for building the **SMPOOS interactive dashboard**, including:
- **System status widgets** (berths occupied, vessels inbound)
- **User activity** (operational load, shifts, worker distribution)
- **Alerts/notification panels** (severity, timestamps, delivery status)
- **Interactive map overlays** (closed berths, hazardous zones, routes)
- **Custom filters** (date, zone, role, alert type)

Dash allows you to implement a functional prototype of the SMPOOS control panel.

10. bootstrap (via Dash Bootstrap Components)
You can use Bootstrap as an external stylesheet for:
- Professional UI layout and theming
- Card components, grids, responsive columns
- Navbar, tabs, and flexible design patterns

Submission

The assessment must be completed individually. You must not share, in part or whole, your assessment with another party other than the module convenor and for the purpose of submission to the university. You must ensure that the University's academic misconduct guidelines are followed in their entirety.

You should use the assessment submission link on the module's Moodle page to submit the following files:

- A **PDF** file for your report. This should not be included in the zip file but instead submitted as a separate file. Failure to do so may result in zero being awarded.
- A **Zip** file of your software artefact. This should contain your software solution and any relevant files to open and execute your solution.

You should ensure that you make a timely submission by the deadline stated at the start of this assessment brief.

Assessment Criteria

Your assessment will be graded according to the following criteria:

Grading criteria	Technical Competence	Problem Solving	Communication Skills
Mark band			
80-100 Distinction	Demonstrates exceptional technical proficiency with a robust, secure, and scalable implementation. Fully meets all functional and non-functional requirements. Exemplary use of object-oriented design, version control, and integration with campus data.	Shows outstanding problem-solving through innovative design choices, effective handling of real-time updates, and advanced analytics. Anticipates and resolves complex challenges with clear rationale.	Produces a highly articulate, well-structured report with strong justification, diagrams, and evaluation. Demonstrates excellent professional writing, referencing, and ethical reflection.
70-79 Distinction	Demonstrates advanced technical skills with a well-executed and mostly complete implementation. Good use of design principles and integration. Minor issues may exist but do not detract from overall quality.	Applies advanced problem-solving strategies with clear logic and rationale. Addresses most challenges effectively and shows good adaptability.	Communicates clearly and professionally. Report is well-organized, with good use of evidence and diagrams. Minor gaps in justification or evaluation may be present.
60-69 Merit	Demonstrates good technical understanding and implementation. Most features are functional and meet requirements. Some inconsistencies or missed opportunities in design or integration.	Shows competent problem-solving with reasonable decisions and partial resolution of challenges. May lack depth in rationale or adaptability.	Report is clear and informative, with appropriate structure and evidence. Some areas may lack depth or clarity, but overall communication is effective.
50-59 Pass (*Threshold*)	Demonstrates basic technical competence. Implementation is functional but may be incomplete or contain flaws. Limited use of design principles or integration.	Problem-solving is evident but lacks sophistication. Some challenges are addressed, but solutions may be simplistic or inconsistent.	Report communicates key ideas but lacks clarity or depth. Structure may be weak, and justification or evaluation may be superficial.
30-49 Fail	Demonstrates limited technical ability. Implementation is incomplete or poorly executed. Major gaps in functionality, design, or integration.	Problem-solving is weak or inconsistent. Struggles to address key challenges or provide rationale.	Report is poorly structured and lacks clarity. Communication is ineffective, with minimal evidence or justification.
0-29 Fail	Demonstrates inadequate technical skills. Unable to produce a functional system. No evidence of design or integration.	No meaningful problem-solving demonstrated. Unable to address basic requirements or challenges.	Report is unclear or missing. Communication is severely deficient, with no evidence or structure.

Learning Outcomes

This assessment will enable students to demonstrate the following learning outcomes as stated in the module outline:

[Module Learning Outcome 2]

- Evaluate and compare principles of software development project management, with an emphasis on agile methodologies and their application in project environments, including project planning, risk management, team coordination, and ethical considerations.

How is this learning outcome addressed?

The assessment brief meets this learning outcome by requiring students to evaluate and apply agile project management principles throughout the software development lifecycle, including planning, risk mitigation (e.g., handling real-time route changes), team coordination, and ethical considerations such as GDPR compliance and accessibility.

[Module Learning Outcome 4]

- Create, design, and thoroughly test software systems to meet specific functional and non-functional requirements, utilising advanced problem-solving, analytical skills, and considering ethical and professional practice.

How is this learning outcome addressed?

The assessment brief aligns with this learning outcome by asking the students to create a of a fully functional software artefact that meets both functional (e.g., real-time navigation, notifications) and non-functional requirements (e.g., scalability, security), supported by rigorous testing and critical evaluation.

[Module Learning Outcome 5]

- Employ Git for source code version control, adeptly handling branches, merging code changes, and resolving conflicts in team-based settings while adhering to industry guidelines.

How is this learning outcome addressed?

This assessment addresses the learning outcome by requiring students to use Git for version control, with evidence of branching, merging, and conflict resolution included in the report and appendices.

[Module Learning Outcome 6]

- Develop and test software using an IDE and an object-oriented programming language, adhering to best practices in code design, readability, and maintainability.

How is this learning outcome addressed?

The assessment brief supports this learning outcome by the use of object-oriented programming and development within an IDE, with an emphasis on maintainable and readable code, as well as modular architecture.

[Module Learning Outcome 7]

- Develop strong self-directed learning skills, actively addressing knowledge gaps, seeking the latest industry information, and reflecting on personal growth, ethical considerations, and research opportunities to advance expertise and professional development.

How is this learning outcome addressed?

The assessment encourages the development of self-directed learning skills by requiring students to actively address knowledge gaps, seek the latest industry information, and reflect on personal growth, ethical considerations, and research opportunities throughout the software development process. This aligns with the outcome's focus on advancing expertise and professional development through reflection and continuous learning.

Regulations, Policies, and Guidelines

Guidance for online submissions
https://www.stmarys.ac.uk/policies/online-submissions.aspx

Academic Misconduct
Any submission must be students' own work and, where facts or ideas have been used from other sources, these sources must be appropriately referenced. Please find a link to the academic misconduct policy below:
https://www.stmarys.ac.uk/policies/academic-regulations.aspx

Ethics Policy
The work being carried out by students must be in compliance with the Ethics Policy. Where there is an ethical issue, as specified within the Ethics Policy, then students will need ethical approval prior to the start of the project. Please find a link to the ethics policy below:
https://www.stmarys.ac.uk/research/students/ethical-review-process.aspx

Extenuating Circumstances
The University's Extenuating Circumstances procedure helps students facing challenges in assessment submission. To request an extension or deferment, submit an EC application with evidence. Approved cases will not incur academic penalties. For longer-term issues, contact Student Services. Please find a link to the EC policy below:
https://www.stmarys.ac.uk/policies/extenuating-circumstances.aspx